

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ

ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ

ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации

ТЕКСТ ПРОГРАММЫ

по расчетно-графической работе
по дисциплине «Программирование»

Выполнил:

студент группы АБ-522

Никулин Р. Д.

Проверил:

(должность)

Архипова А. Б.

Новосибирск 2026

АННОТАЦИЯ

Настоящий документ содержит текст программы «Multi-Algo Cryptotool» и составлен по ГОСТ 19.401-78. Программа разработана в ходе расчетно-графической работы по дисциплине «Программирование» в соответствии с техническим заданием, оформленным по ГОСТ 19.201-78.

Приведены все файлы исходного текста, входящие в состав программного изделия: сценарий сборки, заголовочный файл двоичного интерфейса, исходные тексты общих модулей и основного исполняемого модуля, исходные тексты трех динамических библиотек алгоритмов шифрования, исходный текст тестовой утилиты и сценарии установки и удаления для Linux и для Windows. Всего приведено 27 файлов общим объемом 1541 строка.

Тексты записаны на языке C++ стандарта 2017 года, на языке описания сборки CMake, на языке командного интерпретатора Bash и на языке Windows PowerShell. Каждому файлу предшествует его путь относительно корня репозитория. Текст каждого файла воспроизведен без изменений. Файлы README.md, .gitignore и .clang-format к тексту программы не относятся и не приводятся.

СОДЕРЖАНИЕ

1	Сценарий сборки проекта	5
1.1	CMakeLists.txt	5
2	Заголовочный файл двоичного интерфейса	5
2.1	include/crypto_abi.h	5
3	Общие модули	6
3.1	common/random_bytes.h	6
3.2	common/random_bytes.cpp	6
3.3	common/secure_memory.h	7
3.4	common/secure_memory.cpp	7
4	Основной исполняемый модуль	8
4.1	src/algorithm_registry.h	8
4.2	src/algorithm_registry.cpp	8
4.3	src/byte_io.h	8
4.4	src/byte_io.cpp	9
4.5	src/cli_options.h	10
4.6	src/cli_options.cpp	10
4.7	src/crypto_module.h	12
4.8	src/crypto_module.cpp	13
4.9	src/main.cpp	14
4.10	src/pipeline.h	15
4.11	src/pipeline.cpp	15
4.12	src/platform.h	16
4.13	src/platform.cpp	16
5	Библиотеки алгоритмов шифрования	17
5.1	lib/beaufort/beaufort.cpp	17
5.2	lib/substitution/substitution.cpp	18
5.3	lib/trithemius/trithemius.cpp	19
6	Тестовая утилита	20
6.1	tests/test_algorithms.cpp	20

7	Сценарии установки и удаления	23
7.1	scripts/install.sh	23
7.2	scripts/uninstall.sh	24
7.3	scripts/install.ps1	24
7.4	scripts/uninstall.ps1	25

1 СЦЕНАРИЙ СБОРКИ ПРОЕКТА

1.1 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(cryptum CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)

# Together with the hidden visibility of common this keeps a cipher library exporting
# its five interface functions and nothing else.
set(CMAKE_VISIBILITY_INLINES_HIDDEN ON)

set(CMAKE_BUILD_WITH_INSTALL_RPATH TRUE)
set(CMAKE_INSTALL_RPATH "$ORIGIN")

if(MSVC)
    add_compile_options(/W4 /WX /utf-8)
else()
    add_compile_options(-Wall -Wextra -Wpedantic -Werror)
endif()

add_library(common STATIC common/random_bytes.cpp common/secure_memory.cpp)
target_include_directories(common PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/common
    ${CMAKE_CURRENT_SOURCE_DIR}/include)
set_target_properties(common PROPERTIES
    POSITION_INDEPENDENT_CODE ON
    CXX_VISIBILITY_PRESET hidden)

add_executable(cryptum
    src/main.cpp
    src/cli_options.cpp
    src/byte_io.cpp
    src/platform.cpp
    src/pipeline.cpp
    src/algorithm_registry.cpp
    src/crypto_module.cpp)
target_link_libraries(cryptum PRIVATE common ${CMAKE_DL_LIBS})

add_library(substitution SHARED lib/substitution/substitution.cpp)
target_link_libraries(substitution PRIVATE common)

add_library(beaufort SHARED lib/beaufort/beaufort.cpp)
target_link_libraries(beaufort PRIVATE common)

add_library(trithemius SHARED lib/trithemius/trithemius.cpp)
target_link_libraries(trithemius PRIVATE common)

add_executable(test_algorithms tests/test_algorithms.cpp src/crypto_module.cpp)
target_include_directories(test_algorithms PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}/src)
target_link_libraries(test_algorithms PRIVATE common ${CMAKE_DL_LIBS})
add_dependencies(test_algorithms substitution beaufort trithemius)
```

2 ЗАГОЛОВОЧНЫЙ ФАЙЛ ДВОИЧНОГО ИНТЕРФЕЙСА

2.1 include/crypto_abi.h

```
#ifndef CRYPTUM_CRYPT0_ABI_H
#define CRYPTUM_CRYPT0_ABI_H

#include <stddef.h>
#include <stdint.h>

extern "C"
{
    struct ConstBuffer
    {
        const uint8_t* data;
        size_t size;
    };
};
```

```

struct MutBuffer
{
    uint8_t* data;
    size_t size;
};

struct AlgorithmInfo
{
    const char* algorithm_name;
    size_t key_size;
};

enum OperationType
{
    OPERATION_ENCRYPT = 0,
    OPERATION_DECRYPT = 1
};

enum CryptoStatus
{
    CRYPTO_OK = 0,
    CRYPTO_ERROR_INVALID_KEY = 1,
    CRYPTO_ERROR_INVALID_BUFFER = 2,
    CRYPTO_ERROR_RANDOM_FAILURE = 3,
    CRYPTO_ERROR_INTERNAL = 4
};

// stream_offset extends the interface because a position-dependent cipher cannot be applied
// to a stream chunk by chunk without the absolute index of the first byte of the chunk.
// generate_key extends it because only the library knows which byte sequences form a valid
// key for its algorithm, and the executable must make no assumption about that.

const AlgorithmInfo* get_algorithm_info(void);
size_t get_output_size(size_t input_size, int operation_type);
int generate_key(MutBuffer* key);
int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset);
int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset);
}

#endif

```

3 ОБЩИЕ МОДУЛИ

3.1 common/random_bytes.h

```

#ifndef CRYPTUM_RANDOM_BYTES_H
#define CRYPTUM_RANDOM_BYTES_H

#include <cstdint>
#include <cstddef>

namespace cryptum
{
    void fill_random_bytes(std::uint8_t* data, std::size_t size);
}

#endif

```

3.2 common/random_bytes.cpp

```

#include "random_bytes.h"

#include <stdexcept>

#ifdef _WIN32
# include <bcrypt.h>
# include <windows.h>
#else
# include <cerrno>
# include <sys/random.h>
#endif

namespace cryptum
{

```

```

void fill_random_bytes(std::uint8_t* data, std::size_t size)
{
#ifdef _WIN32
    if (BCryptGenRandom(nullptr, data, static_cast<ULONG>(size), BCRYPT_USE_SYSTEM_PREFERRED_RNG)
        != 0)
        throw std::runtime_error("the secure random source is unavailable");
#else
    std::size_t filled = 0;
    while (filled < size)
    {
        const ssize_t written = getrandom(data + filled, size - filled, 0);
        if (written < 0)
        {
            if (errno == EINTR) continue;
            throw std::runtime_error("the secure random source is unavailable");
        }
        filled += static_cast<std::size_t>(written);
    }
#endif
}
} // namespace cryptum

```

3.3 common/secure_memory.h

```

#ifndef CRYPTUM_SECURE_MEMORY_H
#define CRYPTUM_SECURE_MEMORY_H

#include <cstdint>
#include <cstdint>
#include <vector>

namespace cryptum
{
    void secure_zero(std::uint8_t* data, std::size_t size);

    // Owns a buffer that holds key material or plaintext and wipes it on every exit path.
    struct SecureBuffer
    {
        explicit SecureBuffer(std::size_t size) : bytes(size) {}
        ~SecureBuffer() { secure_zero(bytes.data(), bytes.size()); }

        SecureBuffer(const SecureBuffer&) = delete;
        SecureBuffer& operator=(const SecureBuffer&) = delete;

        std::vector<std::uint8_t> bytes;
    };
} // namespace cryptum

#endif

```

3.4 common/secure_memory.cpp

```

#include "secure_memory.h"

#ifdef _WIN32
#include <windows.h>
#else
#include <string.h>
#endif

namespace cryptum
{
    void secure_zero(std::uint8_t* data, std::size_t size)
    {
#ifdef _WIN32
        SecureZeroMemory(data, size);
#else
        explicit_bzero(data, size);
#endif
    }
} // namespace cryptum

```

4 ОСНОВНОЙ ИСПОЛНЯЕМЫЙ МОДУЛЬ

4.1 src/algorithm_registry.h

```
#ifndef CRYPTUM_ALGORITHM_REGISTRY_H
#define CRYPTUM_ALGORITHM_REGISTRY_H

#include <string>

namespace cryptum
{

std::string supported_algorithms();
std::string library_base_name(const std::string& algorithm);
} // namespace cryptum

#endif
```

4.2 src/algorithm_registry.cpp

```
#include "algorithm_registry.h"

#include <stdexcept>

namespace cryptum
{

namespace
{

struct AlgorithmEntry
{
    const char* algorithm;
    const char* library;
};

const AlgorithmEntry REGISTRY[] = {
    {"substitution", "substitution"},
    {"beaufort", "beaufort"},
    {"trithemius", "trithemius"},
};

} // namespace

std::string supported_algorithms()
{
    std::string names;
    for (const AlgorithmEntry& entry : REGISTRY)
    {
        if (!names.empty()) names += ", ";
        names += entry.algorithm;
    }
    return names;
}

std::string library_base_name(const std::string& algorithm)
{
    for (const AlgorithmEntry& entry : REGISTRY)
        if (algorithm == entry.algorithm) return entry.library;
    throw std::runtime_error("unknown algorithm '" + algorithm + "'; supported algorithms are "
        + supported_algorithms());
}

} // namespace cryptum
```

4.3 src/byte_io.h

```
#ifndef CRYPTUM_BYTE_IO_H
#define CRYPTUM_BYTE_IO_H

#include <cstdint>
#include <fstream>
#include <istream>
#include <ostream>
#include <string>

namespace cryptum
```

```

{

std::istream& open_input(const std::string& path, std::ifstream& file);
std::ostream& open_output(const std::string& path, std::ofstream& file);

std::size_t read_bytes(std::istream& stream, std::uint8_t* data, std::size_t size);
void        write_bytes(std::ostream& stream, const std::uint8_t* data, std::size_t size);
void        read_key(const std::string& path, const std::string& algorithm_name, std::uint8_t* data,
                    std::size_t size);
void        write_key(const std::string& path, const std::uint8_t* data, std::size_t size);
} // namespace cryptum

#endif

```

4.4 src/byte_io.cpp

```

#include "byte_io.h"

#include <filesystem>
#include <iostream>
#include <stdexcept>
#include <string>

namespace cryptum
{

std::istream& open_input(const std::string& path, std::ifstream& file)
{
    if (path == "-") return std::cin;
    file.open(path, std::ios::binary);
    if (!file) throw std::runtime_error("cannot open input file '" + path + "'");
    return file;
}

std::ostream& open_output(const std::string& path, std::ofstream& file)
{
    if (path == "-") return std::cout;
    file.open(path, std::ios::binary | std::ios::trunc);
    if (!file) throw std::runtime_error("cannot open output file '" + path + "'");
    return file;
}

std::size_t read_bytes(std::istream& stream, std::uint8_t* data, std::size_t size)
{
    stream.read(reinterpret_cast<char*>(data), static_cast<std::streamsize>(size));
    if (stream.bad()) throw std::runtime_error("reading the input failed");
    return static_cast<std::size_t>(stream.gcount());
}

void write_bytes(std::ostream& stream, const std::uint8_t* data, std::size_t size)
{
    stream.write(reinterpret_cast<const char*>(data), static_cast<std::streamsize>(size));
    if (!stream) throw std::runtime_error("writing the output failed");
}

void read_key(const std::string& path, const std::string& algorithm_name, std::uint8_t* data,
             std::size_t size)
{
    std::ifstream file;
    std::istream& stream = open_input(path, file);

    // One byte beyond the expected length is read on purpose: it tells a key file that is too
    // long from one of exactly the right size, without seeking in a stream that may be a pipe.
    std::uint8_t beyond = 0;
    if (read_bytes(stream, data, size) != size || read_bytes(stream, &beyond, 1) != 0)
        throw std::runtime_error("the key for " + algorithm_name + " must be exactly "
                                + std::to_string(size) + " bytes long");
}

void write_key(const std::string& path, const std::uint8_t* data, std::size_t size)
{
    std::ofstream file;
    std::ostream& stream = open_output(path, file);

    // The permissions are narrowed while the file is still empty, so the key is never
    // readable by anyone else, not even for the time of the write.
    if (file.is_open())
        std::filesystem::permissions(
            path, std::filesystem::perms::owner_read | std::filesystem::perms::owner_write,
            std::filesystem::perm_options::replace);

    write_bytes(stream, data, size);
    stream.flush();
}


```

```

    if (!stream) throw std::runtime_error("writing the key failed");
}
} // namespace cryptum

```

4.5 src/cli_options.h

```

#ifndef CRYPTUM_CLI_OPTIONS_H
#define CRYPTUM_CLI_OPTIONS_H

#include <string>

namespace cryptum
{
    enum ProgramMode
    {
        MODE_HELP,
        MODE_GENERATE_KEY,
        MODE_ENCRYPT,
        MODE_DECRYPT
    };

    struct CliOptions
    {
        ProgramMode mode = MODE_HELP;
        std::string algorithm;
        std::string key_path;
        std::string input_path;
        std::string output_path;
        std::string save_key_path;
        bool generate_key = false;
        bool has_key = false;
        bool has_input = false;
        bool has_output = false;
        bool has_save_key = false;
    };

    CliOptions parse_command_line(int argument_count, char** arguments);
} // namespace cryptum

#endif

```

4.6 src/cli_options.cpp

```

#include "cli_options.h"

#include <cstdint>
#include <stdexcept>
#include <vector>

namespace cryptum
{
    namespace
    {
        std::string take_value(const std::string& option, const std::string& attached_value,
                               bool has_attached_value, const std::vector<std::string>& arguments,
                               std::size_t& index)
        {
            if (has_attached_value) return attached_value;
            if (index + 1 >= arguments.size())
                throw std::runtime_error("option " + option + " requires an argument");
            ++index;
            return arguments[index];
        }

        void reject_attached_value(const std::string& option, bool has_attached_value)
        {
            if (has_attached_value) throw std::runtime_error("option " + option + " takes no argument");
        }

        ProgramMode parse_mode(const std::string& name)
        {
            if (name == "encrypt") return MODE_ENCRYPT;
            if (name == "decrypt") return MODE_DECRYPT;
            if (name == "generate-key") return MODE_GENERATE_KEY;
            throw std::runtime_error("unknown mode '" + name
                                      + "'; expected encrypt, decrypt or generate-key");
        }
    }
}

```

```

bool requests_help(const std::vector<std::string>& arguments)
{
    for (const std::string& argument : arguments)
        if (argument == "-h" || argument == "--help") return true;
    return false;
}

void validate(const CliOptions& options)
{
    if (options.mode == MODE_GENERATE_KEY)
    {
        if (options.has_input) throw std::runtime_error("--input is not used in generate-key mode");
        if (options.has_output)
            throw std::runtime_error("--output is not used in generate-key mode");
        if (options.has_key) throw std::runtime_error("--key is not used in generate-key mode");
        if (options.generate_key)
            throw std::runtime_error("--generate-key is not used in generate-key mode");
        if (!options.has_save_key)
            throw std::runtime_error("generate-key mode requires --save-key or --write-key");
        return;
    }

    if (options.has_key && options.generate_key)
        throw std::runtime_error("--key and --generate-key cannot be combined");
    if (!options.has_key && !options.generate_key)
        throw std::runtime_error("encryption and decryption require --key or --generate-key");

    if (options.mode == MODE_DECRYPT)
    {
        if (options.generate_key)
            throw std::runtime_error("--generate-key cannot be used in decrypt mode");
        if (options.has_save_key)
            throw std::runtime_error("--save-key and --write-key cannot be used in decrypt mode");
    }

    if (options.generate_key && !options.has_save_key)
        throw std::runtime_error("--generate-key requires --save-key or --write-key, otherwise the"
                                   " generated key is lost irrecoverably");

    const bool input_from_standard_input = !options.has_input || options.input_path == "-";
    if (input_from_standard_input && options.has_key && options.key_path == "-")
        throw std::runtime_error("the data and the key cannot both be read from standard input");

    const bool output_to_standard_output = !options.has_output || options.output_path == "-";
    if (output_to_standard_output && options.has_save_key && options.save_key_path == "-")
        throw std::runtime_error("the data and the key cannot both be written to standard output");
}

} // namespace

CliOptions parse_command_line(int argument_count, char** arguments)
{
    const std::vector<std::string> given(arguments + 1, arguments + argument_count);

    CliOptions options;
    if (given.empty() || requests_help(given)) return options;

    bool algorithm_given = false;
    bool mode_given = false;
    std::string mode_name;

    for (std::size_t index = 0; index < given.size(); ++index)
    {
        const std::string& argument = given[index];

        std::string name = argument;
        std::string attached_value;
        bool has_attached_value = false;

        const std::size_t separator = argument.find('=');
        if (argument.rfind("--", 0) == 0 && separator != std::string::npos)
        {
            name = argument.substr(0, separator);
            attached_value = argument.substr(separator + 1);
            has_attached_value = true;
        }

        if (name == "-a" || name == "--algorithm")
        {
            options.algorithm = take_value(name, attached_value, has_attached_value, given, index);
            algorithm_given = true;
        }
        else if (name == "-m" || name == "--mode")
        {
            mode_name = take_value(name, attached_value, has_attached_value, given, index);
            mode_given = true;
        }
    }
}

```

```

    }
    else if (name == "-k" || name == "--key")
    {
        options.key_path = take_value(name, attached_value, has_attached_value, given, index);
        options.has_key = true;
    }
    else if (name == "-i" || name == "--input")
    {
        options.input_path = take_value(name, attached_value, has_attached_value, given, index);
        options.has_input = true;
    }
    else if (name == "-o" || name == "--output")
    {
        options.output_path
            = take_value(name, attached_value, has_attached_value, given, index);
        options.has_output = true;
    }
    else if (name == "-g" || name == "--generate-key")
    {
        reject_attached_value(name, has_attached_value);
        options.generate_key = true;
    }
    else if (name == "-s" || name == "--save-key")
    {
        options.save_key_path
            = take_value(name, attached_value, has_attached_value, given, index);
        options.has_save_key = true;
    }
    else if (name == "-w" || name == "--write-key")
    {
        reject_attached_value(name, has_attached_value);
        options.save_key_path = "-";
        options.has_save_key = true;
    }
    else if (!name.empty() && name[0] == '-')
    {
        throw std::runtime_error("unknown option '" + name + "'");
    }
    else { throw std::runtime_error("unexpected positional argument '" + argument + "'"); }
}

if (!algorithm_given) throw std::runtime_error("missing required option --algorithm");
if (!mode_given) throw std::runtime_error("missing required option --mode");
options.mode = parse_mode(mode_name);
validate(options);

if (options.mode != MODE_GENERATE_KEY)
{
    if (!options.has_input) options.input_path = "-";
    if (!options.has_output) options.output_path = "-";
}

return options;
}
} // namespace cryptum

```

4.7 src/cryptum_module.h

```

#ifndef CRYPTUM_CRYPTUM_MODULE_H
#define CRYPTUM_CRYPTUM_MODULE_H

#include "crypto_abi.h"

#include <cstddef>
#include <cstdint>
#include <string>

namespace cryptum
{
    // Owns the dynamic library handle so that it is released on every exit path.
    struct CryptoModule
    {
        CryptoModule() = default;
        ~CryptoModule();

        CryptoModule(const CryptoModule&) = delete;
        CryptoModule& operator=(const CryptoModule&) = delete;

        void* handle = nullptr;

        const AlgorithmInfo* (*get_algorithm_info)() = nullptr;
        std::size_t (*get_output_size)(std::size_t, int) = nullptr;
    };
}

```

```

    int (*generate_key)(MutBuffer*) = nullptr;
    int (*encrypt)(ConstBuffer, ConstBuffer, MutBuffer*, std::uint64_t) = nullptr;
    int (*decrypt)(ConstBuffer, ConstBuffer, MutBuffer*, std::uint64_t) = nullptr;
};

void load_module(CryptoModule& module, const std::string& library_base_name);
} // namespace cryptum

#endif

```

4.8 src/crypto_module.cpp

```

#include "crypto_module.h"

#include <stdexcept>

#ifdef _WIN32
# include <windows.h>
#else
# include <dlfcn.h>
#endif

namespace cryptum
{
    namespace
    {
        using GenericFunction = void (*)();

        std::string library_file_name(const std::string& base_name)
        {
            #ifdef _WIN32
                return base_name + ".dll";
            #else
                return "lib" + base_name + ".so";
            #endif
        }

        void* open_library(const std::string& file_name)
        {
            #ifdef _WIN32
                void* handle = LoadLibraryA(file_name.c_str());
            #else
                void* handle = dlopen(file_name.c_str(), RTLD_NOW | RTLD_LOCAL);
            #endif
            if (handle == nullptr)
                throw std::runtime_error("cannot load the algorithm library '" + file_name + "'");
            return handle;
        }

        GenericFunction resolve(void* handle, const char* symbol)
        {
            #ifdef _WIN32
                GenericFunction function
                    = reinterpret_cast<GenericFunction>(GetProcAddress(static_cast<HMODULE>(handle), symbol));
            #else
                GenericFunction function = reinterpret_cast<GenericFunction>(dlsym(handle, symbol));
            #endif
            if (function == nullptr)
                throw std::runtime_error("the algorithm library does not export '" + symbol
                                          + "'");
            return function;
        }
    } // namespace

    CryptoModule::~CryptoModule()
    {
        if (handle == nullptr) return;
        #ifdef _WIN32
            FreeLibrary(static_cast<HMODULE>(handle));
        #else
            dlclose(handle);
        #endif
    }

    void load_module(CryptoModule& module, const std::string& library_base_name)
    {
        module.handle = open_library(library_file_name(library_base_name));

        module.get_algorithm_info = reinterpret_cast<decltype(module.get_algorithm_info)>(
            resolve(module.handle, "get_algorithm_info"));
        module.get_output_size = reinterpret_cast<decltype(module.get_output_size)>(

```

```

        resolve(module.handle, "get_output_size"));
module.generate_key
    = reinterpret_cast<decltype(module.generate_key)>(resolve(module.handle, "generate_key"));
module.encrypt = reinterpret_cast<decltype(module.encrypt)>(resolve(module.handle, "encrypt"));
module.decrypt = reinterpret_cast<decltype(module.decrypt)>(resolve(module.handle, "decrypt"));
}
} // namespace cryptum

```

4.9 src/main.cpp

```

#include "algorithm_registry.h"
#include "byte_io.h"
#include "cli_options.h"
#include "crypto_module.h"
#include "pipeline.h"
#include "platform.h"

#include <exception>
#include <iostream>
#include <stdexcept>

namespace
{
void print_help()
{
    std::cout << "Usage: cryptum --algorithm NAME --mode MODE [options]\n"
                "\n"
                "Options:\n"
                "  -h, --help           print this help and exit\n"
                "  -a, --algorithm NAME algorithm to use\n"
                "  -m, --mode MODE      encrypt, decrypt or generate-key\n"
                "  -k, --key PATH       read the key from PATH, '-' for standard input\n"
                "  -i, --input PATH     read the data from PATH, '-' or omitted for "
                "standard input\n"
                "  -o, --output PATH    write the data to PATH, '-' or omitted for "
                "standard output\n"
                "  -g, --generate-key   generate a fresh key and use it for this operation\n"
                "  -s, --save-key PATH  write the key in use to PATH, '-' for standard "
                "output\n"
                "  -w, --write-key      equivalent to --save-key -\n"
                "\n"
                "An option takes its argument as '--option value', '--option=value' or\n"
                "'-o value'; combined short options and attached short values are not "
                "supported.\n"
                "\n"
                "Algorithms:\n"
                " "
                << cryptum::supported_algorithms() << "\n";
}

void run(const cryptum::CliOptions& options)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, cryptum::library_base_name(options.algorithm));

    const AlgorithmInfo* info = module.get_algorithm_info();
    if (info == nullptr || info->algorithm_name == nullptr || info->key_size == 0)
        throw std::runtime_error("the algorithm library reports no usable metadata");

    cryptum::SecureBuffer key(info->key_size);
    if (options.mode == cryptum::MODE_GENERATE_KEY || options.generate_key)
        cryptum::run_key_generation(module.generate_key, key);
    else
        cryptum::read_key(options.key_path, info->algorithm_name, key.bytes.data(),
                          key.bytes.size());

    if (options.has_save_key)
        cryptum::write_key(options.save_key_path, key.bytes.data(), key.bytes.size());
    if (options.mode == cryptum::MODE_GENERATE_KEY) return;

    std::ifstream input_file;
    std::ofstream output_file;
    std::istream& input = cryptum::open_input(options.input_path, input_file);
    std::ostream& output = cryptum::open_output(options.output_path, output_file);

    const OperationType operation
        = options.mode == cryptum::MODE_ENCRYPT ? OPERATION_ENCRYPT : OPERATION_DECRYPT;
    cryptum::run_transformation(operation == OPERATION_ENCRYPT ? module.encrypt : module.decrypt,
                                module.get_output_size, operation,
                                ConstBuffer{key.bytes.data(), key.bytes.size()}, input, output);
}
} // namespace

```

```

int main(int argument_count, char** arguments)
{
    try
    {
        cryptum::initialise_platform();

        const cryptum::CliOptions options = cryptum::parse_command_line(argument_count, arguments);
        if (options.mode == cryptum::MODE_HELP) print_help();
        else run(options);
        return 0;
    }
    catch (const std::exception& error)
    {
        std::cerr << "cryptum: " << error.what() << "\n";
        return 1;
    }
}

```

4.10 src/pipeline.h

```

#ifndef CRYPTUM_PIPELINE_H
#define CRYPTUM_PIPELINE_H

#include "crypto_abi.h"
#include "secure_memory.h"

#include <cstdint>
#include <cstdint>
#include <istream>
#include <ostream>

namespace cryptum
{
    using GenerateKeyFunction = int (*)(MutBuffer*);
    using OutputSizeFunction = std::size_t (*)(std::size_t, int);
    using TransformFunction = int (*)(ConstBuffer, ConstBuffer, MutBuffer*, std::uint64_t);

    void run_key_generation(GenerateKeyFunction generate, SecureBuffer& key);
    void run_transformation(TransformFunction transform, OutputSizeFunction output_size,
                           OperationType operation, ConstBuffer key, std::istream& input,
                           std::ostream& output);
} // namespace cryptum

#endif

```

4.11 src/pipeline.cpp

```

#include "pipeline.h"
#include "byte_io.h"

#include <stdexcept>
#include <string>

namespace cryptum
{
    namespace
    {
        constexpr std::size_t CHUNK_SIZE = 64 * 1024;

        void check_status(int status, const std::string& operation)
        {
            switch (status)
            {
                case CRYPTO_OK:
                    return;
                case CRYPTO_ERROR_INVALID_KEY:
                    throw std::runtime_error(operation + " failed: the key is not valid for this algorithm");
                case CRYPTO_ERROR_INVALID_BUFFER:
                    throw std::runtime_error(operation
                                             + " failed: the buffer passed to the algorithm is too"
                                             + " small");
                case CRYPTO_ERROR_RANDOM_FAILURE:
                    throw std::runtime_error(operation + " failed: the secure random source is unavailable");
                default:
                    throw std::runtime_error(operation
                                             + " failed: the algorithm library reported an internal"

```

```

        " error");
    }
} // namespace

void run_key_generation(GenerateKeyFunction generate, SecureBuffer& key)
{
    MutBuffer target{key.bytes.data(), key.bytes.size()};
    check_status(generate(&target), "key generation");
    if (target.size != key.bytes.size())
        throw std::runtime_error("key generation failed: the algorithm produced a key of"
                                   " unexpected length");
}

void run_transformation(TransformFunction transform, OutputSizeFunction output_size,
                        OperationType operation, ConstBuffer key, std::istream& input,
                        std::ostream& output)
{
    const std::string operation_name = operation == OPERATION_ENCRYPT ? "encryption" : "decryption";

    SecureBuffer input_chunk(CHUNK_SIZE);
    SecureBuffer output_chunk(output_size(CHUNK_SIZE, operation));
    std::uint64_t stream_offset = 0;

    for (;;)
    {
        const std::size_t taken = read_bytes(input, input_chunk.bytes.data(), CHUNK_SIZE);
        if (taken == 0) break;

        const ConstBuffer source{input_chunk.bytes.data(), taken};
        MutBuffer target{output_chunk.bytes.data(), output_chunk.bytes.size()};
        check_status(transform(key, source, &target, stream_offset), operation_name);
        write_bytes(output, target.data, target.size);

        stream_offset += taken;
    }

    output.flush();
    if (!output) throw std::runtime_error("writing the output failed");
}
} // namespace cryptum

```

4.12 src/platform.h

```

#ifndef CRYPTUM_PLATFORM_H
#define CRYPTUM_PLATFORM_H

namespace cryptum
{

void initialise_platform();

}

#endif

```

4.13 src/platform.cpp

```

#include "platform.h"

#ifdef _WIN32
# include <cstdio>
# include <fcntl.h>
# include <io.h>
# include <windows.h>
#endif

namespace cryptum
{

void initialise_platform()
{
#ifdef _WIN32
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
    _setmode(_fileno(stdin), _O_BINARY);
    _setmode(_fileno(stdout), _O_BINARY);
#endif
}

} // namespace cryptum

```

5 БИБЛИОТЕКИ АЛГОРИТМОВ ШИФРОВАНИЯ

5.1 lib/beaufort/beaufort.cpp

```
#include "crypto_abi.h"
#include "random_bytes.h"

#include <algorithm>
#include <cstdint>
#include <cstdlib>
#include <stdexcept>

namespace
{
    constexpr std::size_t KEY_SIZE = 16;

    const AlgorithmInfo ALGORITHM_INFO = {"beaufort", KEY_SIZE};

    bool is_valid_key(ConstBuffer key) { return key.data != nullptr && key.size == KEY_SIZE; }
} // namespace

extern "C"
{
    const AlgorithmInfo* get_algorithm_info(void) { return &ALGORITHM_INFO; }

    size_t get_output_size(size_t input_size, int) { return input_size; }

    // Every sequence of sixteen bytes is a valid key, so the random bytes are used as they come.
    int generate_key(MutBuffer* key)
    {
        try
        {
            if (key == nullptr || key->data == nullptr || key->size < KEY_SIZE)
                return CRYPTO_ERROR_INVALID_BUFFER;

            cryptum::fill_random_bytes(key->data, KEY_SIZE);
            key->size = KEY_SIZE;
            return CRYPTO_OK;
        }
        catch (const std::runtime_error&)
        {
            return CRYPTO_ERROR_RANDOM_FAILURE;
        }
        catch (...)
        {
            return CRYPTO_ERROR_INTERNAL;
        }
    }

    //  $C[i] = K[i \bmod 16] - P[i] \pmod{256}$ , where  $i$  is the absolute index of the byte in the
    // stream; it is taken from the address of the byte so that the lambda needs no state.
    int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)
    {
        try
        {
            if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
            if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
            if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

            std::transform(input.data, input.data + input.size, output->data,
                [key, input, stream_offset](const std::uint8_t& value) {
                    const std::uint64_t index
                        = stream_offset
                          + static_cast<std::uint64_t>(&value - input.data);
                    return static_cast<std::uint8_t>(key.data[index % KEY_SIZE] - value);
                });
            output->size = input.size;
            return CRYPTO_OK;
        }
        catch (...)
        {
            return CRYPTO_ERROR_INTERNAL;
        }
    }

    // The transformation is an involution, so decryption is the very same operation.
    int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)
    {
        return encrypt(key, input, output, stream_offset);
    }
}
```

5.2 lib/substitution/substitution.cpp

```
#include "crypto_abi.h"
#include "random_bytes.h"
#include "secure_memory.h"

#include <algorithm>
#include <array>
#include <cstdint>
#include <stdint>
#include <limits>
#include <numeric>
#include <random>
#include <stdexcept>

namespace
{
    constexpr std::size_t KEY_SIZE = 256;

    const AlgorithmInfo ALGORITHM_INFO = {"simple substitution", KEY_SIZE};

    bool is_valid_key(ConstBuffer key)
    {
        if (key.data == nullptr || key.size != KEY_SIZE) return false;

        std::array<std::uint8_t, KEY_SIZE> occurrences{};
        std::for_each(key.data, key.data + key.size,
            [&occurrences](std::uint8_t value) { ++occurrences[value]; });
        return std::all_of(occurrences.begin(), occurrences.end(),
            [](std::uint8_t count) { return count == 1; });
    }

    // A uniform random bit generator is what std::shuffle expects; this one draws from the
    // system random source instead of a seeded pseudo random engine.
    struct SecureRandomEngine
    {
        using result_type = std::uint32_t;

        static constexpr result_type min() { return 0; }
        static constexpr result_type max() { return std::numeric_limits<result_type>::max(); }

        result_type operator()() const
        {
            result_type value = 0;
            cryptum::fill_random_bytes(reinterpret_cast<std::uint8_t*>(&value), sizeof(value));
            return value;
        }
    };

    void apply_table(const std::uint8_t* table, ConstBuffer input, MutBuffer* output)
    {
        std::transform(input.data, input.data + input.size, output->data,
            [table](std::uint8_t value) { return table[value]; });
        output->size = input.size;
    }
} // namespace

extern "C"
{
    const AlgorithmInfo* get_algorithm_info(void) { return &ALGORITHM_INFO; }

    size_t get_output_size(size_t input_size, int) { return input_size; }

    int generate_key(MutBuffer* key)
    {
        try
        {
            if (key == nullptr || key->data == nullptr || key->size < KEY_SIZE)
                return CRYPTO_ERROR_INVALID_BUFFER;

            std::iota(key->data, key->data + KEY_SIZE, std::uint8_t{0});
            std::shuffle(key->data, key->data + KEY_SIZE, SecureRandomEngine{});
            key->size = KEY_SIZE;
            return CRYPTO_OK;
        }
        catch (const std::runtime_error&)
        {
            return CRYPTO_ERROR_RANDOM_FAILURE;
        }
        catch (...)
        {
            return CRYPTO_ERROR_INTERNAL;
        }
    }
}
```

```

// The substitution is position independent, so the stream offset is not used here.
int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t)
{
    try
    {
        if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
        if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
        if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

        apply_table(key.data, input, output);
        return CRYPTO_OK;
    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}

int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t)
{
    try
    {
        if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
        if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
        if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

        cryptum::SecureBuffer inverse(KEY_SIZE);
        for (std::size_t value = 0; value < KEY_SIZE; ++value)
            inverse.bytes[key.data[value]] = static_cast<std::uint8_t>(value);

        apply_table(inverse.bytes.data(), input, output);
        return CRYPTO_OK;
    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}
}

```

5.3 lib/trithemius/trithemius.cpp

```

#include "crypto_abi.h"
#include "random_bytes.h"

#include <algorithm>
#include <cstdint>
#include <stdint>
#include <stdexcept>

namespace
{
    constexpr std::size_t KEY_SIZE = 16;

    const AlgorithmInfo ALGORITHM_INFO = {"trithemius", KEY_SIZE};

    bool is_valid_key(ConstBuffer key) { return key.data != nullptr && key.size == KEY_SIZE; }

    std::uint64_t absolute_index(const std::uint8_t& value, ConstBuffer input, uint64_t stream_offset)
    {
        return stream_offset + static_cast<std::uint64_t>(&value - input.data);
    }
} // namespace

extern "C"
{
    const AlgorithmInfo* get_algorithm_info(void) { return &ALGORITHM_INFO; }

    size_t get_output_size(size_t input_size, int) { return input_size; }

    // Every sequence of sixteen bytes is a valid key, so the random bytes are used as they come.
    int generate_key(MutBuffer* key)
    {
        try
        {
            if (key == nullptr || key->data == nullptr || key->size < KEY_SIZE)
                return CRYPTO_ERROR_INVALID_BUFFER;

            cryptum::fill_random_bytes(key->data, KEY_SIZE);
            key->size = KEY_SIZE;
            return CRYPTO_OK;
        }
        catch (...)
        {
            return CRYPTO_ERROR_INTERNAL;
        }
    }
}

```

```

    }
    catch (const std::runtime_error&)
    {
        return CRYPTO_ERROR_RANDOM_FAILURE;
    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}

// C[i] = P[i] + K[i mod 16] + i (mod 256), where i is the absolute index of the byte in the
// stream; it is taken from the address of the byte so that the lambda needs no state.
int encrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)
{
    try
    {
        if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
        if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
        if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

        std::transform(
            input.data, input.data + input.size, output->data,
            [key, input, stream_offset](const std::uint8_t& value) {
                const std::uint64_t index = absolute_index(value, input, stream_offset);
                return static_cast<std::uint8_t>(value + key.data[index % KEY_SIZE] + index);
            });
        output->size = input.size;
        return CRYPTO_OK;
    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}

int decrypt(ConstBuffer key, ConstBuffer input, MutBuffer* output, uint64_t stream_offset)
{
    try
    {
        if (output == nullptr) return CRYPTO_ERROR_INVALID_BUFFER;
        if (!is_valid_key(key)) return CRYPTO_ERROR_INVALID_KEY;
        if (output->size < input.size) return CRYPTO_ERROR_INVALID_BUFFER;

        std::transform(
            input.data, input.data + input.size, output->data,
            [key, input, stream_offset](const std::uint8_t& value) {
                const std::uint64_t index = absolute_index(value, input, stream_offset);
                return static_cast<std::uint8_t>(value - key.data[index % KEY_SIZE] - index);
            });
        output->size = input.size;
        return CRYPTO_OK;
    }
    catch (...)
    {
        return CRYPTO_ERROR_INTERNAL;
    }
}
}

```

6 ТЕСТОВАЯ УТИЛИТА

6.1 tests/test_algorithms.cpp

```

#include "crypto_module.h"

#include <algorithm>
#include <cstdint>
#include <cstdlib>
#include <iostream>
#include <iterator>
#include <random>
#include <stdexcept>
#include <string>
#include <vector>

namespace
{
    using Bytes = std::vector<std::uint8_t>;
}

```

```

using TransformFunction = int (*)(ConstBuffer, ConstBuffer, MutBuffer*, std::uint64_t);

constexpr std::uint32_t SEED = 20260915;

struct TestCase
{
    const char* name;
    void (*run)();
};

void require(bool condition, const std::string& message)
{
    if (!condition) throw std::runtime_error(message);
}

Bytes pseudo_random_bytes(std::size_t size)
{
    std::mt19937 engine(SEED);
    Bytes data(size);
    std::generate(data.begin(), data.end(),
                  [&engine] { return static_cast<std::uint8_t>(engine()); });
    return data;
}

Bytes apply(TransformFunction transform, const Bytes& key, const Bytes& input,
            std::uint64_t stream_offset)
{
    Bytes output(input.size());
    MutBuffer target{output.data(), output.size()};
    const int status = transform(ConstBuffer{key.data(), key.size()},
                                ConstBuffer{input.data(), input.size()}, &target, stream_offset);
    require(status == CRYPTO_OK, "the algorithm returned status " + std::to_string(status));
    output.resize(target.size());
    return output;
}

Bytes generated_key(const cryptum::CryptoModule& module)
{
    const AlgorithmInfo* info = module.get_algorithm_info();
    Bytes key(info->key_size);
    MutBuffer target{key.data(), key.size()};
    require(module.generate_key(&target) == CRYPTO_OK, "the key could not be generated");
    require(target.size() == info->key_size, "the generated key has the wrong length");
    return key;
}

void check_round_trip(const char* library, const Bytes& plaintext)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, library);

    const Bytes key = generated_key(module);
    const Bytes ciphertext = apply(module.encrypt, key, plaintext, 0);
    require(ciphertext.size() == plaintext.size(), "the ciphertext has a different length");
    require(apply(module.decrypt, key, ciphertext, 0) == plaintext,
            "the decrypted data differs from the original");
}

void check_known_answer(const char* library, const Bytes& key, const Bytes& plaintext,
                        const Bytes& ciphertext)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, library);

    require(apply(module.encrypt, key, plaintext, 0) == ciphertext,
            "the ciphertext differs from the expected bytes");
    require(apply(module.decrypt, key, ciphertext, 0) == plaintext,
            "the recovered plaintext differs from the expected bytes");
}

Bytes reversing_key()
{
    Bytes key(256);
    for (std::size_t value = 0; value < key.size(); ++value)
        key[value] = static_cast<std::uint8_t>(255 - value);
    return key;
}

Bytes sixteen_byte_key()
{
    return {0xA0, 0xB1, 0xC2, 0xD3, 0xE4, 0xF5, 0x06, 0x17,
            0x28, 0x39, 0x4A, 0x5B, 0x6C, 0x7D, 0x8E, 0x9F};
}

// The expected bytes come from the formulas themselves: the substitution key maps b to
// 255 - b, beaufort computes K[i mod 16] - P[i] and trithemius P[i] + K[i mod 16] + i.

```

```

void substitution_known_answer()
{
    check_known_answer("substitution", reversing_key(), {0x00, 0x01, 0x41, 0xFF},
        {0xFF, 0xFE, 0xBE, 0x00});
}

void beaufort_known_answer()
{
    check_known_answer("beaufort", sixteen_byte_key(), {0x10, 0x20, 0x30, 0x41, 0x05},
        {0x90, 0x91, 0x92, 0x92, 0xDF});
}

void trithemius_known_answer()
{
    check_known_answer("trithemius", sixteen_byte_key(), {0x10, 0x20, 0x30, 0x41, 0x05},
        {0xB0, 0xD2, 0xF4, 0x17, 0xED});
}

void substitution_empty_input() { check_round_trip("substitution", Bytes()); }
void beaufort_empty_input() { check_round_trip("beaufort", Bytes()); }
void trithemius_empty_input() { check_round_trip("trithemius", Bytes()); }

void substitution_unaligned_input() { check_round_trip("substitution", pseudo_random_bytes(37)); }
void beaufort_unaligned_input() { check_round_trip("beaufort", pseudo_random_bytes(37)); }
void trithemius_unaligned_input() { check_round_trip("trithemius", pseudo_random_bytes(37)); }

void substitution_one_mebibyte()
{
    check_round_trip("substitution", pseudo_random_bytes(1024 * 1024));
}
void beaufort_one_mebibyte() { check_round_trip("beaufort", pseudo_random_bytes(1024 * 1024)); }
void trithemius_one_mebibyte() { check_round_trip("trithemius", pseudo_random_bytes(1024 * 1024)); }

void check_chunked_matches_single_call(const char* library)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, library);

    const Bytes key = generated_key(module);
    const Bytes plaintext = pseudo_random_bytes(1000);
    const Bytes whole = apply(module.encrypt, key, plaintext, 0);

    const std::size_t CHUNKS[] = {1, 7, 13, 64, 300, 615};
    Bytes chunked;
    std::size_t offset = 0;
    for (std::size_t size : CHUNKS)
    {
        const Bytes part(plaintext.begin() + static_cast<std::ptrdiff_t>(offset),
            plaintext.begin() + static_cast<std::ptrdiff_t>(offset + size));
        const Bytes encrypted = apply(module.encrypt, key, part, offset);
        chunked.insert(chunked.end(), encrypted.begin(), encrypted.end());
        offset += size;
    }

    require(offset == plaintext.size(), "the chunks do not cover the whole input");
    require(chunked == whole, "chunk by chunk encryption differs from a single call");
}

void check_key_generation(const char* library)
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, library);

    const AlgorithmInfo* info = module.get_algorithm_info();
    require(generated_key(module).size() == info->key_size,
        "the generated key does not have the declared length");

    Bytes too_small(info->key_size - 1);
    MutBuffer target(too_small.data(), too_small.size());
    require(module.generate_key(&target) == CRYPTO_ERROR_INVALID_BUFFER,
        "a buffer smaller than the key length was accepted");
}

void substitution_chunks() { check_chunked_matches_single_call("substitution"); }
void beaufort_chunks() { check_chunked_matches_single_call("beaufort"); }
void trithemius_chunks() { check_chunked_matches_single_call("trithemius"); }

void substitution_key_generation() { check_key_generation("substitution"); }
void beaufort_key_generation() { check_key_generation("beaufort"); }
void trithemius_key_generation() { check_key_generation("trithemius"); }

void substitution_rejects_invalid_keys()
{
    cryptum::CryptoModule module;
    cryptum::load_module(module, "substitution");
}

```

```

const Bytes      plaintext = {0x41, 0x42, 0x43};
Bytes            output(plaintext.size());
MutBuffer        target{output.data(), output.size()};
const ConstBuffer source{plaintext.data(), plaintext.size()};

const Bytes repeated(256, 0x00);
require(module.encrypt(ConstBuffer{repeated.data(), repeated.size()}, source, &target, 0)
        == CRYPTO_ERROR_INVALID_KEY,
        "a key that is not a permutation was accepted");

const Bytes wrong_length(16, 0x00);
require(
    module.encrypt(ConstBuffer{wrong_length.data(), wrong_length.size()}, source, &target, 0)
        == CRYPTO_ERROR_INVALID_KEY,
    "a key of the wrong length was accepted");
}

const TestCase TESTS[] = {
    {"substitution: known answer", substitution_known_answer},
    {"substitution: empty input", substitution_empty_input},
    {"substitution: length that is not a multiple of the key length", substitution_unaligned_input},
    {"substitution: one mebibyte of pseudo random data", substitution_one_mebibyte},
    {"substitution: chunk by chunk equals a single call", substitution_chunks},
    {"substitution: generated key length", substitution_key_generation},
    {"beaufort: known answer", beaufort_known_answer},
    {"beaufort: empty input", beaufort_empty_input},
    {"beaufort: length that is not a multiple of the key length", beaufort_unaligned_input},
    {"beaufort: one mebibyte of pseudo random data", beaufort_one_mebibyte},
    {"beaufort: chunk by chunk equals a single call", beaufort_chunks},
    {"beaufort: generated key length", beaufort_key_generation},
    {"trithemius: known answer", trithemius_known_answer},
    {"trithemius: empty input", trithemius_empty_input},
    {"trithemius: length that is not a multiple of the key length", trithemius_unaligned_input},
    {"trithemius: one mebibyte of pseudo random data", trithemius_one_mebibyte},
    {"trithemius: chunk by chunk equals a single call", trithemius_chunks},
    {"trithemius: generated key length", trithemius_key_generation},
    {"substitution: a key that is not a permutation is rejected",
     substitution_rejects_invalid_keys},
};
} // namespace

int main()
{
    const int total    = static_cast<int>(std::size(TESTS));
    int         failures = 0;

    for (const TestCase& test : TESTS)
    {
        try
        {
            test.run();
            std::cout << "pass  " << test.name << "\n";
        }
        catch (const std::exception& error)
        {
            std::cout << "FAIL  " << test.name << ": " << error.what() << "\n";
            ++failures;
        }
    }

    std::cout << total - failures << " of " << total << " tests passed\n";
    return failures == 0 ? 0 : 1;
}

```

7 СЦЕНАРИИ УСТАНОВКИ И УДАЛЕНИЯ

7.1 scripts/install.sh

```

#!/usr/bin/env bash
set -euo pipefail

LIBRARY_DIRECTORY=/usr/local/lib/cryptum
EXECUTABLE_LINK=/usr/local/bin/cryptum
LIBRARIES=(libsubstitution.so libbeaufort.so libtrithemius.so)

script_directory=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)
source_directory=${1:-$script_directory/./build/bin}

if [ "${id -u}" -ne 0 ]; then
    echo "install.sh: administrative rights are required" >&2

```

```

    exit 1
fi

for file in cryptum "${LIBRARIES[@]"}; do
    if [ ! -f "$source_directory/$file" ]; then
        echo "install.sh: '$source_directory/$file' not found, build the project first" >&2
        exit 1
    fi
done

install -d -m 0755 "$LIBRARY_DIRECTORY"
install -m 0755 "$source_directory/cryptum" "$LIBRARY_DIRECTORY/cryptum"
for file in "${LIBRARIES[@]"}; do
    install -m 0644 "$source_directory/$file" "$LIBRARY_DIRECTORY/$file"
done
ln -sf "$LIBRARY_DIRECTORY/cryptum" "$EXECUTABLE_LINK"

echo "cryptum installed in $LIBRARY_DIRECTORY and linked as $EXECUTABLE_LINK"

```

7.2 scripts/uninstall.sh

```

#!/usr/bin/env bash
set -euo pipefail

LIBRARY_DIRECTORY=/usr/local/lib/cryptum
EXECUTABLE_LINK=/usr/local/bin/cryptum
LIBRARIES=(libsubstitution.so libbeaufort.so libtrithemius.so)

if [ "$(id -u)" -ne 0 ]; then
    echo "uninstall.sh: administrative rights are required" >&2
    exit 1
fi

if [ -L "$EXECUTABLE_LINK" ] && [ "$(readlink "$EXECUTABLE_LINK")" = "$LIBRARY_DIRECTORY/cryptum" ]; then
    rm -f "$EXECUTABLE_LINK"
fi

for file in cryptum "${LIBRARIES[@]"}; do
    rm -f "$LIBRARY_DIRECTORY/$file"
done
rmdir "$LIBRARY_DIRECTORY" 2>/dev/null || true

echo "cryptum removed"

```

7.3 scripts/install.ps1

```

param([string]$SourceDirectory = (Join-Path $PSScriptRoot '..\build\bin'))

$ErrorActionPreference = 'Stop'

$InstallDirectory = Join-Path $env:LOCALAPPDATA 'cryptum'
$Files = @('cryptum.exe', 'substitution.dll', 'beaufort.dll', 'trithemius.dll')

foreach ($file in $Files) {
    if (-not (Test-Path (Join-Path $SourceDirectory $file))) {
        throw "'$file' not found in '$SourceDirectory', build the project first"
    }
}

New-Item -ItemType Directory -Force -Path $InstallDirectory | Out-Null
foreach ($file in $Files) {
    Copy-Item -Force -Path (Join-Path $SourceDirectory $file) -Destination $InstallDirectory
}

$userPath = [Environment]::GetEnvironmentVariable('Path', 'User')
if ([string]::IsNullOrEmpty($userPath)) {
    [Environment]::SetEnvironmentVariable('Path', $InstallDirectory, 'User')
}
elseif ($userPath.Split(';') -notcontains $InstallDirectory) {
    [Environment]::SetEnvironmentVariable('Path', "$userPath;$InstallDirectory", 'User')
}

Write-Output "cryptum installed in $InstallDirectory"

```

7.4 scripts/uninstall.ps1

```
$ErrorActionPreference = 'Stop'

$InstallDirectory = Join-Path $env:LOCALAPPDATA 'cryptum'

if (Test-Path $InstallDirectory) {
    Remove-Item -Recurse -Force -Path $InstallDirectory
}

$UserPath = [Environment]::GetEnvironmentVariable('Path', 'User')
if (-not [string]::IsNullOrEmpty($UserPath)) {
    $entries = $UserPath.Split(';') | Where-Object { $_ -ne $InstallDirectory -and $_ -ne '' }
    [Environment]::SetEnvironmentVariable('Path', ($entries -join ';'), 'User')
}

Write-Output 'cryptum removed'
```